

LSF 通用模擬邏輯規範 (Universal LSF Simulation Logic)

1. 符號定義 (Notation)

我們定義一個 Fault Primitive $FP = \langle S_0, S, F, R_{\{det\}} \rangle$:

- S_0 : 致敏前必須具備的記憶體狀態 (Initial State Requirement)。
- S : 致敏寫入序列 (Sensitizing Sequence), 長度為 L 。 $S = [w_1, w_2, \dots, w_L]$ 。
- F : 故障值 (Fault Effect)。
- $R_{\{det\}}$: 偵測值 (Detection Value)。

系統狀態變數 (State Variables) 在時間點 t :

- $V_t \in \{0, 1\}$: 當前 Cell 的物理值 (Physical Value)。
- $k_t \in \{0, 1, \dots, L-1\}$: 當前致敏序列的匹配進度 (Match Index)。
 - $k_t = 0$ 表示尚未開始匹配。
 - $k_t = i$ 表示已經連續匹配了 S 的前 i 個動作。

2. 狀態轉移函數 (Transition Functions)

對於每一個 March Operation Op_t , 我們依序計算 k_{t+1} (進度更新) 和 V_{t+1} (物理值更新)。

Rule 1: 中斷規則 (The Breaker Rule)

適用於非寫入操作或邊界: $Op_t \in \{R(d), C(d), \text{End of Element}\}$ 。

這些操作會強制重置致敏進度, 且不改變物理值。

$$\begin{cases} k_{t+1} = 0 \\ V_{t+1} = V_t \end{cases} \quad \text{Status} = \text{Check Detection (if R or C)}$$

- 推論: 因為 V 不變, 所以 C 或 S 無法 Mask 故障; 因為 k 歸零, 所以它們會打斷累積。

Rule 2: 寫入匹配規則 (The Write Matching Rule)

適用於寫入操作: $Op_t = W(d)$ 。

我們首先計算暫定進度 (Tentative Index) k' 。這必須同時滿足「數值匹配」與「初始條件」。

定義 KMP 的 NextState 函數：

$k' = \text{KMP_Transition}(k_t, d, S)$

(直觀解釋：若 d 接得上 $S[k_t+1]$ 則 $k' = k_t + 1$ ；否則根據 *Failure Function* 回退)

嚴格約束 (Constraint):

若 $k' = 1$ (即剛開始匹配序列的第一個字)，必須檢查當前物理值是否符合初始條件 S_0 。

$k_{\text{next}} = \begin{cases} k' & \text{if } k' > 1 \\ 1 & \text{if } k' = 1 \text{ AND } V_t = S_0 \\ \text{otherwise (Mismatch or } S_0 \text{ unsatisfied)} \end{cases}$

Rule 3: 觸發與遮蔽規則 (The Trigger vs. Mask Rule)

根據 k_{next} 的結果決定物理值 V_{t+1} 。

情況 A: 致敏完成 (Trigger)

若 $k_{\text{next}} == L$ (匹配長度等於序列長度):

$\begin{cases} V_{t+1} = F & (\text{Force Fault Value}) \\ k_{t+1} = \text{KMP_Overlap}(S) & (\text{Handle Overlapping Patterns}) \end{cases}$

- 解釋：這是故障觸發。物理值被強制覆蓋為 F 。

情況 B: 未完成/普通寫入 (Normal Write / Mask)

若 $k_{\text{next}} < L$:

$\begin{cases} V_{t+1} = d & (\text{Write Data Value}) \\ k_{t+1} = k_{\text{next}} \end{cases}$

- 解釋：這是 Masking 的數學本質。若 V_t 原本是 F (故障中)，而 $d \neq F$ ，則 V_{t+1} 變回 d ，故障被修復。

3. 邏輯流程圖 (Logic Flow) for Exam Solving

在解題時，對每一個操作 Op 執行此迴圈：

- Is Breaker?** ($R, C, ;$)
 - Yes $\rightarrow k=0$, V 不變。(若為 R/C 則檢查 V 是否等於預期值)。 **Done.**
- Is Write?** ($W(d)$)
 - 計算 KMP 下一步 k_{next} 。
 - Check S_0 :** 如果這是序列的第一步 ($k_{\text{next}}=1$)，檢查 $V_{\text{curr}} == S_0$?
 - 不符合 $\rightarrow k_{\text{next}}=0$ 。
- Update V:**
 - 若 $k_{\text{next}} == \text{Length}(S)$ $\rightarrow$ **Trigger!** $V = F$ 。

- 若 $k_{next} < \text{Length}(S)$ $\rightarrow$ **Write/Mask!** $V = d$ 。

4. 經典案例驗證 (Validation)

我們用這套嚴格邏輯來跑你剛才的兩大難題。

Case A: 單步致敏陷阱

$FP = <0W1/0/->$, $S = \{W1\}$, $L = 1$, $S_0 = 0$, $F = 0$ 。

題目: $a(W0, W1, W1, R1)$

- **Step 1: W0**
 - Write 0。 k 歸 0。 $V = 0$ 。
- **Step 2: W1**
 - $Op = W1$ 。 S 的第 1 個字是 $W1$ $\rightarrow$ 匹配。
 - 檢查 S_0 : 當前 $V = 0$ 。符合 $S_0 = 0$ 。
 - $k_{next} = 1$ 。
 - 判定: $k_{next} == L$ ($1 == 1$)。 **Trigger!**
 - $V \leftarrow 0$ 。
- **Step 3: W1 (關鍵)**
 - $Op = W1$ 。 S 的第 1 個字是 $W1$ $\rightarrow$ 匹配。
 - 檢查 S_0 : 當前 $V = 0$ (因上一步 Trigger)。符合 $S_0 = 0$ 。
 - $k_{next} = 1$ 。
 - 判定: $k_{next} == L$ 。 **Trigger!** (Re-trigger)
 - $V \leftarrow 0$ 。(而非 Write 1)
- **Step 4: R1**
 - 讀到 0。 Fault。

Case B: C 的 Masking 區別

$FP = <0W0W1/0/->$, $S = \{W0, W1\}$, $L = 2$, $S_0 = 0$ 。

題目 1: $W0, C, W1$ (C 插在中間)

- **W0:** $k = 1$, $V = 0$ (符合 S_0)。
- **C: Breaker Rule** $\rightarrow k = 0$, $V = 0$ (保持)。
- **W1:**
 - $Op = W1$ 。嘗試接續? k 已是 0, 無法接 $W0$ to $W1$ 。
 - 嘗試當開頭? $S[1]$ 是 $W0$, 但 Op 是 $W1$ 。不匹配。
 - $k_{next} = 0$ 。
 - 判定: Normal Write。
 - $V \leftarrow 1$ 。(Masking 若之前有錯, 這裡變正)

題目 2: $W0, W1, C$ (C 插在後面)

- **W0:** $k=1, V=0$.
- **W1:** $k=2, V=0$ (**Trigger**).
- **C: Breaker Rule** $\rightarrow k=0, V=0$.
 - 注意: Breaker Rule 說 $V_{t+1} = V_t$ 。
 - 所以 V 保持 0 (Fault)。
 - 結果: Fault 被保留 (C 不會 Mask)。

5. 給考生的快速口訣 (Cheat Sheet)

1. 看到 **R, C**, 分號 $\rightarrow$ 進度全砍, 數值照搬。(KMP Reset, Value Keep)
2. 看到 **Write** $\rightarrow$ 先算進度, 再定數值。
 - 進度滿了? $\rightarrow$ 變 **Fault** (Trigger)。
 - 進度沒滿? $\rightarrow$ 變 **Data** (可能 Mask)。
3. 單步致敏 (**W1**) $\rightarrow$ 看前身。
 - 前身符合 $S_0 \rightarrow$ 它是 Trigger (寫不進去)。
 - 前身不符 $S_0 \rightarrow$ 它是 Mask (寫得進去)。